CONTENTS

1 INTRODUCTION 1
- 1.1 Research Questions 1
- 1.2 Scope and Limitations 2
- 1.3 Related Work 2

2 BACKGROUND 3
- 2.1 Traditional Graph Representation Learning 3
  - 2.1.1 Transductive Nature of Traditional GRL 4
- 2.2 Neural Networks 4
  - 2.2.1 Weight Optimization 5
- 2.3 Graph Neural Networks 5
  - 2.3.1 Graph Neural Network Architecture 6
- 2.4 Graph Neural Networks and Large Graphs 7
  - 2.4.1 Training on Large Datasets Under Memory Constraints 7
  - 2.4.2 Subgraph Sampling 8
- 2.5 PyTorch Geometric 9
- 2.6 Neo4j 9
- 2.7 Evaluation Metrics 10

3 METHODOLOGY 11
- 3.1 System Architecture 11
- 3.2 Graph Representation in Neo4j 11
- 3.3 PyG Interface Implementations 12
- 3.4 Sampling Algorithms 12
- 3.5 Graph Neural Network Model 12
- 3.6 Datasets and Experimental Setup 13
- 3.7 Benchmarking Methodology 13
  - 3.7.1 End-to-End Latency Decomposition 13
  - 3.7.2 Query Execution Profiling 13
  - 3.7.3 Python-Level Profiling 14
  - 3.7.4 Sampler Comparison Framework 14
- 3.8 Correctness and Verification 14

A APPENDIX 15

BIBLIOGRAPHY 17

LIST OF FIGURES

LIST OF TABLES

LISTINGS

ACRONYMS

1 INTRODUCTION

Graph structures can be helpful as a tool in modeling several important domains. Examples of applications of such models are found in domains as different as that of predicting likely chemical bonds between atoms within medical science, and the problem of fraud detection in the field of finance. The prevalence of graph-structured data across many domains creates a need for efficient and accurate methods for modeling and analyzing graphs.

GNNs have emerged as a leading approach for representation learning on graphs, combining the structural intuition behind classical graph algorithms with the expressiveness and inductivity of neural networks. Unlike traditional graph representation learning (GRL) algorithms, GNNs are inductive in nature, and unlike most traditional neural networks they do not treat datapoints independently of other inputs. This allows GNNs to capture graph structure as well as generalize to unseen nodes, making them highly effective when analyzing graph-structured data. However, the graph structure also introduces a new challenge absent from traditional neural networks: the challenge of growing training data. The inter-connectivity between datapoints in a graph makes it impossible to consider datapoints independently, as is typically assumed in stochastic gradient descent (SGD).

A natural solution is to store the graph in a dedicated graph database and query it during training for the relevant neighbors of nodes in a mini-batch. Graph databases are especially built for efficient graph traversal and feature look-up—precisely the operations needed when training GNNs on large graphs. Yet there have been few studies exploring synergies between GNNs and graph databases.

This thesis explores whether graph databases can be used as a scalable and efficient solution for training GNNs on large graphs. We build upon the `PyTorch Geometric` machine learning library and couple it with subgraph fetching from `Neo4j`, where the graph data resides.

1.1 RESEARCH QUESTIONS

This thesis addresses the following research questions:

1. **RQ1 — Bottlenecks:** What are the primary bottlenecks in GNN training when using a graph database for graph storage, and to what extent can they be mitigated?

2. RQ2 — **Sampling:** Which subgraph sampling strategies are most efficient when the graph is stored in a graph database?

3. RQ3 — **Graph size:** For which graph sizes does a graph-database-based storage approach provide the greatest benefits?

1.2 SCOPE AND LIMITATIONS

The experiments in this thesis are conducted on four datasets of varying sizes using a two-layer Graph Convolutional Network (GCN) for node classification. The main emphasis is on the training pipeline, rather than on a particular downstream task or model architecture. Since the core stages of the training pipeline remain broadly similar across multiple graph learning tasks, the conclusions may have relevance beyond the specific task considered here.

1.3 RELATED WORK

Prior work on combining GNN training with graph databases is limited. The main directly relevant work is by Lopushanskyy and Shi, who show that a GNN can be trained directly on a graph database by retrieving only the required subgraphs and features into memory during training. Their results indicate that graph databases can serve as both graph store and feature store, and can support both single-machine and distributed training.

Most previous work on scaling GNN training instead relies on distributed systems built around graph partitioning. ByteGNN improves distributed mini-batch training through GNN-tailored graph partitioning, parallel sampling, and scheduling strategies that increase throughput. In contrast, G3 focuses on scalable full-graph training, where workers exchange intermediate representations in a pipelined manner to avoid duplicate computation across partitions.

Compared with these systems, our work studies a different point in the design space: using a graph database as the primary graph storage and sampling backend, and evaluating under what conditions this approach is efficient for GNN training.

2 BACKGROUND

Graphs in mathematics are used to model pairwise relationships between objects. Objects are modeled as nodes $\mathcal{V}$, and their relations are called edges $\mathcal{E}$. Many domains benefit from modeling data as objects (nodes) and relationships (edges). Consider, for example, social networks, protein–drug interactions, or railway networks. Studying graph structure and node features allows us to predict properties of the graph. For example, predictions may concern how a graph evolves over time. In fraud detection, nodes can represent accounts and edges transactions. By modeling typical graph evolution, such as which transactions and accounts are added, it becomes possible to detect anomalous transactions that deviate from expected patterns.

When making predictions about graphs it is often good to start by creating an embedding of the graph. The optimal embedding depends on the downstream task. There are several ways to embed a graph, transductively with traditional graph representation learning (GRL) approaches, and inductively with GNNs.

In this chapter we hope to give a fundamental understanding in topics such as traditional GRL algorithms, neural networks, describe how GNNs combine ideas from traditional graph representation learning and neural networks. From traditional GRL they inherit the idea that node embeddings should reflect graph structure, while from neural networks they adopt a learnable, parameterized function. We also bring up challenges working with GNNs on big datasets, that don't exist in traditional neural networks.

Lastly we hope to give a basic introduction in `PyTorch Geometric`, the machine learning library we use during experiments, and `Neo4j`, a graph database that provides useful graph operations for training GNNs, which is used as the graph storage during the experiments.

2.1 TRADITIONAL GRAPH REPRESENTATION LEARNING

In graph representation learning we aim to find a representation of the graph that encodes it's structure. Embeddings like this are often used in downstream tasks such as node classification, link prediction, or clustering. Finding a good embedding is somewhat dependent on what the downstream task is.

The simplest way of constructing node embeddings is to manually choose a node statistic to encode the node (e.g. node degree or centrality measures). A more advanced group of methods involve factorizing the adjacency matrix into node-node similarity. Another class

of methods is based on random walks. These methods learn embeddings by maximizing co-occurrence between nodes appearing in the same random walks, and can be shown to implicitly perform matrix factorization.

2.1.1 *Transductive Nature of Traditional GRL*

A common feature of all GRL algorithms discussed above—whether using node statistics, matrix factorization, or random walks—is that no weights are shared between nodes: no embedding-generating function is learned, only node embeddings for the current nodes in the graph. If new nodes are introduced, not only is there no way to compute their embeddings, the embeddings of the existing nodes also become inaccurate (the latter also occurs if nodes are removed).

Consider a simplified version of the optimization problem for random walk algorithms:

$$\min_{\{z_x \,:\, x \in \mathcal{V}\}} \sum_{(u,v)\in\mathcal{D}} -\log\Big(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)\Big)\,,$$

where $\mathcal{D}$ is the set of node pairs encountered on random walks, and $\mathcal{V}$ is the set of nodes in the graph. The optimization problem above optimizes the actual node embeddings directly, rather than some embedding-generating function. This is what makes all traditional representation learning algorithms *transductive* in nature.

Consider a social network in which people are nodes and their relationships are edges. If a transductive graph representation is used, we learn feature vectors for specific people in the graph. If more people are introduced to the network, we must recompute every person's feature vector.

2.2 NEURAL NETWORKS

In contrast to the transductive nature of traditional GRL, neural networks are *inductive*: they learn a generalizable function.

Neural networks are models composed of interconnected processing units called neurons. How neurons are connected depends on the model architecture. Some neurons receive external input and pass it further into the network; others aggregate information from preceding neurons; and output neurons produce the final result. Every connection between two neurons is associated with a weight, by which the activation of the upstream neuron is scaled before reaching the next neuron. The network learns patterns by adjusting these weights.

The neural network can be defined as a function of weights θ that takes an arbitrary input x and outputs $\hat{y}$:

$$f(x\,,\,\theta) = \hat{y}.$$

The network learns by optimizing θ to minimize a loss function $\mathcal{L}$ over training data X, which quantifies the prediction error—either supervised (given a desired output y) or unsupervised (without a ground-truth outcome):

$$\min_{\theta} \mathcal{L}(X, \theta) = \sum_{x \in X} l\,(\,f(x, \theta)\,,\,y\,). \qquad \text{(supervised)}$$

The training data consists of input datapoints $\{x \in X_{\text{train}}\}$ mapped to targets $\{y \in Y_{\text{train}}\}$, forming the training set $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$. The idea is to maximize the likelihood $p(y_i \mid x_i, \theta)$ of the network producing the correct output for each input by adjusting the weights, in the hope that the learned weights generalize to unseen data. The model's generalization ability is evaluated on a separate test set drawn from the same distribution as the training data.

2.2.1 *Weight Optimization*

The optimization method used to find the optimal weights is typically some variant of gradient descent (GD). GD involves iteratively updating the model weights based on the current gradient of the loss function. Differentiating the loss with respect to the weights $\nabla_\theta \mathcal{L}$ gives the direction along which the loss increases most steeply; hence the negative gradient indicates the direction of steepest descent.[1] The weights are updated in proportion to the gradient with step size η, commonly called the learning rate:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \cdot \nabla_{\theta^{(k)}} \mathcal{L}.$$

Datapoints are typically assumed to be independent in standard supervised learning, meaning each prediction $\hat{y}_i$ depends only on x_i. Under this assumption the full gradient is a sum over all datapoints, making vanilla gradient descent computationally expensive. Stochastic gradient descent (SGD) approximates this gradient using a uniformly sampled mini-batch, reducing computation per update. Although SGD does not guarantee a decrease in loss at every step, its expected gradient is an unbiased estimate of the full-batch gradient, yielding convergence in expectation under the independence assumption.

2.3 GRAPH NEURAL NETWORKS

Graph neural networks incorporate ideas from traditional graph representation learning by preserving the intuition that node representations should reflect the structure of the graph, i.e., nodes that are close

1 GD converges to the global minimizer for convex differentiable functions with appropriate step sizes. The loss function for neural networks is generally not convex, but variants of gradient descent are still used to find "good" weights.

or structurally similar should have similar embeddings. However, instead of relying on fixed similarity measures or directly optimizing node embeddings, GNNs use a neural network architecture to learn this relationship.

By combining this structural inductive bias with a parameterized, trainable function, GNNs learn to generate node embeddings through neighborhood aggregation. This allows the model to adapt the representation to the downstream task using gradient-based optimization, rather than relying on manually designed features or predefined objectives. As a result, GNNs are more expressive and flexible than traditional GRL methods. At the same time, they have a fundamentally different architecture: instead of learning embeddings tied to specific nodes in a fixed graph, they learn a shared function that can generate embeddings for any node based on its features and local neighborhood.

Consider the social network example from section 2.1. When a new person enters the social network, an embedding can be generated directly using the learned embedding function. Unlike transductive methods, there is no need to retrain the model or recompute embeddings for existing nodes. The same function can also be applied to an entirely new social network, as long as the underlying data distribution is similar to the one seen during training. This ability to generalize to unseen nodes and graphs is what makes GNNs inductive.

2.3.1 *Graph Neural Network Architecture*

The assumption of independence between datapoints is fundamentally violated in graph-structured data. In a graph, each node is explicitly defined through its relationships to other nodes, therefore considering nodes in isolation looses lots of information. The label $\hat{y}_v$ or representation h_v of a node v is often highly dependent on its neighborhood structure and on the attributes of nearby nodes. Ignoring these dependencies discards essential information.

GNNs address this limitation by explicitly modeling relational dependencies. Instead of learning a function where the output for a object (node) is calculated solely with its own features $f(h_v, \theta) = \hat{y}_i$, GNNs learn representations that incorporate information from a node's local neighborhood in the graph $f(v \cup \mathcal{N}(v), \theta) = \hat{y}_i$, allowing predictions to depend on both node attributes and the structure of their connections.

GNNs are built from stacked message passing layers. At each message passing layer k, a node v updates its representation $\mathbf{x}_v^{(k)}$ by aggregating information from its neighbors:

$$\mathbf{x}_v^{(k)} = \phi^{(k)}\left(\mathbf{x}_v^{(k-1)}, \bigoplus_{u \in \mathcal{N}(v)} \psi^{(k)}\left(\mathbf{x}_v^{(k-1)}, \mathbf{x}_u^{(k-1)}\right)\right).$$

- $\psi^{(k)}$ is neighborhood aggregation function. It takes the feature vectors of the neighbors and outputs a vector representing the neigborhood. $\psi^{(k)}$ is usuallt permutation invariant of the neighbors to avoid encoding bias in the order of neighbors.
- $\phi^{(k)}$ takes as input the feature vector $\mathbf{x}_v^{(k)}$ and output of $\psi^{(k)}$, and outputs the updated feature vector $\mathbf{x}_v^{(k)}$.

After K layers, the final node representation encodes information about it's K-hop neighborhood. This iterative message passing enables the GNN to capture long range dependencies between K-hop neighboors, inaccesible to standard neural networks.

2.4 GRAPH NEURAL NETWORKS AND LARGE GRAPHS

In typical neural networks, datapoints are either independent or have dependencies with a clear and structured form. This structure is determined by the architectural inductive bias, which makes it straightforward to derive which datapoints influence each other. For example, in RNNs, t_n depends on $\{t_{n-1}, t_{n-2}, \dots\}$, while in convolutional neural networks (CNNs), a pixel $p_{x,y}$ depends only on a local neighborhood $\{\, p_{i,j} \mid |i-x| \leqslant r,\ |j-y| \leqslant r \,\}$ in the same image. In both cases, the set of dependent datapoints is explicitly defined and bounded.

In contrast, graph neural networks lack such explicit structural assumptions. The dependency structure is determined by the graph itself, making it harder to determine which datapoints influence a given node. Moreover, these dependencies are not only less structured, but also significantly larger. The representation of a node depends on its k-hop neighborhood, which can grow exponentially with k. As a result, each datapoint in a GNN may depend on orders of magnitude more datapoints than in models such as CNNs, where dependencies are limited to small local regions.

2.4.1 *Training on Large Datasets Under Memory Constraints*

In standard neural networks, the independence (or structured-bounded dependence) between datapoints allows training on datasets larger

than available memory. Data can be processed in virtual mini-batches $\{d^{(1)}, \ldots, d^{(n)}\}$ such that

$$\sum_i d_x^{(i)} < Y,$$

where Y is the available memory. Crucially, the gradient computed on a batch depends only on the datapoints within that batch. This assumption breaks for GNNs as the output for one node is dependent on an un-bounded number of neighbors. The gradient for the seed nodes in the batch can't be computed accuraretely without their neighbors. To solve this the neighbors of the seed nodes are fetched in the beginning of every batch, to build a sub-graph around the seed nodes so that the gradient for that batch can be computed accuretly.

Algorithmus 1 : Mini-batch GNN Training (Single Epoch)

```
// Mini-batch training with neighbor-sampled subgraphs
Input : f_θ, V
Output : θ
for each batch B ⊆ V do
    𝒢_B ← ∅
    for each seed node v ∈ B do
        (X_v, y_v, E_v) ← SAMPLE(v)
        𝒢_B ← 𝒢_B ∪ (X_v, y_v, E_v)
    ŷ ← f_θ(X_B, E_B)
    ℒ ← CROSSENTROPY(ŷ, y_B)
    θ ← θ − η∇_θ ℒ
return θ
```

2.4.2 *Subgraph Sampling*

The un-structured and un-bounded node-node dependence makes it impossible to consider the full k-hop neighborhoods for nodes in a batch, because of copmutation and memory constraints.

To solve this a fast bounded neighborhood sampling algorithm is needed. Upper bounds are usually defined as $[k_0, k_1, .., k_n]$ where k_i defines the maximum number of neighbors to be sampled at that level k. If the number of neighbors exceeds the bound, then neighbors are usually sampled uniformly at random without replacement.

Some sampling algorithms like the one used in graphsage introduce bias in the GNN output. The expected neighborhood aggregation is the same for sampling in graphsage. But bias is introduced when the expected neighboorhood aggregation is inserted in a non-linear activation function, in the update step. GNNs have shown to converge well enough even though this bias, one recent is because GNNs are usually relatively shallow with commonly 2 layers.

2.5 PYTORCH GEOMETRIC

PyTorch Geometric is a graph neural network library that enables easy creation of GNNs in Python. It is built on top of PyTorch and provides helpful modules such as predefined message-passing layers (graph attention, graph convolution, and more), making it straightforward to construct customizable GNNs by combining different layers within implementations of the abstract PyTorch `nn.Module` class.

PyTorch Geometric also provides a framework for storing the graph remotely in a database, through the following abstract classes:

- `FeatureStore`: Defines how node feature vectors are stored and fetched. A fast feature-vector look-up is essential.
- `GraphStore`: Defines how the graph topology is stored. In contrast to the `FeatureStore`, the emphasis here is on fast traversal rather than fast look-up, since efficient neighbor sampling requires rapid graph traversal.
- `BaseSampler`: Allows the user to define a sampling method for selecting neighbors used in the forward pass. The `BaseSampler` is tightly coupled with the `GraphStore`. Multiple sampling processes can be spawned; since instances of the sampler are recreated in each separate process, it is important not to store large amounts of data in the sampler.
- `NodeLoader`: Takes implementations of `FeatureStore`, `GraphStore`, and `BaseSampler` together with batching hyperparameters (e. g., batch size) and provides a data loader that splits the data into mini-batches, samples a subgraph for each batch, and fetches the corresponding node feature vectors from the `FeatureStore`.

2.6 NEO4J

Neo4j is a persistence layer used in diverse applications where graph data modeling is of importance. More concretely, Neo4j is an ACID-compliant database management system. Neo4j's query language is called Cypher and has been influential in developing a standardized Graph Query Language (GQL).[2]

A Cypher string is parsed into an Abstract Syntax Tree (AST), which is then validated—raising an error, for example, if a non-existent label, relationship type, or property filter predicate is used. The validated AST is normalized into canonical form and subsequently used to generate one or more logical plans. A logical plan is an operator tree that specifies how to produce the query response.

As in a standard algebraic expression, operators in the tree are evaluated bottom-up according to precedence. Leaf operators access the

2 ISO/IEC 39075

data source directly, using primitives such as `NodeByLabelScan` (retrieve all nodes with a given label) or `Filter` (apply a predicate such as `n.age` > 30). Higher operators combine or transform these results. The physical execution plan is derived from the logical plan by the query planner, which selects concrete index structures and join strategies to minimize execution cost.

2.7 EVALUATION METRICS

Throughout the experiments a set of metrics is used to evaluate the system implementation:

- **Accuracy.** Node classification accuracy on the held-out test set, used to verify that the graph-database-backed pipeline converges to the same quality as an in-memory baseline.
- **Latency and throughput.** Per-batch wall-clock time and the number of batches processed per second, decomposed into sub-phases: DB execution, Python ETL, and GNN compute.
- **End-to-end latency breakdown.** A fine-grained decomposition of the per-batch latency into driver-level components: time to first result row (t_{first}), total server-side latency (t_{last}), network and driver receive time, and Python ETL overhead.
- **Query execution profiling.** Operator-level statistics (rows processed, DB hits, memory) obtained via the Cypher `PROFILE` keyword, used to diagnose query plan inefficiencies.
- **Python-level profiling.** Call-stack profiles collected with `cProfile`, used to identify hot-spots in the Python data pipeline.

3 METHODOLOGY

This research was conducted at Neo4j, within the Graph Data Science team. The goal was to build and evaluate a GNN training pipeline that uses Neo4j as the primary graph store.

The first step was to implement a naive GNN training system coupled with a graph database. This was compared with a PyTorch Geometric in-memory implementation to verify that the graph-database backend did not degrade model quality, i. e.that it did not cause slower convergence or a worse final test accuracy. Training throughput was not compared between the two implementations at this stage, since the PyG baseline keeps the entire graph in RAM while the graph-database implementation stores the majority of the graph on disk.

3.1 SYSTEM ARCHITECTURE

The system consists of three main components: a Neo4j graph database acting as the graph and feature store, a Python training process that implements the PyG abstract interfaces, and a `NodeLoader` that orchestrates mini-batch construction during training. The implementations `FeatureStore` and `GraphStore` acts as the interface between the training process and the `Neo4j` database. The implementation of `BaseSampler` defines how the subgraph sampling is being done.

3.2 GRAPH REPRESENTATION IN NEO4J

Nodes in Neo4j are stored with a label corresponding to the node type (e. g., `Paper` for citation networks). Each node carries the following properties:

- A unique integer id.
- Pre-computed node feature vectors. The vectors can either be stored as Float64[] or bytes[]
- A target class label for supervised node classification, which is an integer signifying what class the node belongs to
- A split indicator (`train`, `val`, `test`) used to partition the dataset. This can either be a string, or integer.

Edges are stored as typed relationships connecting pairs of nodes. No edge features are used in the experiments.

3.3 PyG Interface Implementations

We implement the three PyG abstract classes described in Section 2.5:

`Neo4jAbstractFS` is an abstract class extending FeatureStore. The purpose of this class is to provide storage of the feature vectors (e.g. a graph database), and to implement an efficient interface for fetching feature vectors. The reason it is kept abstract is that while there are some methods any Neo4j interacting PyG FeatureStore needs (e.g. executing queries) there are also some features that may vary depending on the use case. One example of this is caching. Avoiding the neo4j fetching by storing commonly used features in RAM is a good optimization, but how this caching is done in the feature store is something that should be up to the user. There are also two extensions of the `Neo4jAbstractFS`, which in the `Neo4jCachedFS` and `Neo4jNoCacheFS`.

`Neo4jAbstractGS` an abstract class extending GraphStore, with the main goal of implementing efficient traversal over the graph to enable fast subgraph sampling. It has two implementations `Neo4jMultiGS` and `Neo4jSingleGS` where the difference is how many sampler they are optimized of being able to handle.

`Neo4jSampler` implements the sampling algorithm, and is tightly coupled with the graph store where it executes the sampling algorithm.

3.4 Sampling Algorithms

Two sampling strategies are evaluated:

NEIGHBOR SAMPLING. For each seed node in the mini-batch, the sampler issues a Cypher query that traverses the graph up to K hops, sampling at most $[k_0, k_1, \ldots, k_{K-1}]$ neighbors at each hop. Neighbors exceeding the limit are selected uniformly at random without replacement. This corresponds to the GraphSAGE sampling strategy.

GRAPHSAINT RANDOM-WALK SAMPLING. Instead of expanding neighborhoods around seed nodes, GraphSAINT constructs a subgraph by first sampling a set of nodes via random walks and then inducing the subgraph over those nodes.

3.5 Graph Neural Network Model

All experiments use a two-layer Graph Convolutional Network (GCN). The model applies two `GCNConv` message-passing layers with ReLU activations and a final linear classifier. The architecture is intentionally

kept simple to focus evaluation on the data pipeline rather than the model itself.

3.6 DATASETS AND EXPERIMENTAL SETUP

Experiments are conducted on the following datasets:

All experiments are run on a single machine with the Neo4j database instance running locally. Hyperparameters such as learning rate, number of epochs, batch size, and sampling limits are kept consistent across compared configurations and recorded in a central configuration file.

3.7 BENCHMARKING METHODOLOGY

A dedicated `Measurer` class wraps the training loop and collects timing, profiling, and resource utilization data for each run, by logging events to a csv file. the csv file is then compiled into plots, and other result files summarizing the experiment outcome

3.7.1 *End-to-End Latency Decomposition*

Per-batch wall-clock time is decomposed into the following segments:

- **DB server latency (topology / features).** The total time from query submission to consumption of the last result row (t_{last}), as reported by the Neo4j Bolt protocol summary. This includes query planning, execution, serialization, and network round-trip time. The time to the first result row (t_{first}) is indicated as a darker sub-segment within this bar.
- **Network + driver receive.** The additional client-side time spent receiving and deserializing Bolt protocol packets after the first row arrives, computed as client wall time minus t_{last}.
- **Python ETL.** Time spent constructing PyG data structures from raw query results (index remapping, tensor assembly).
- **GNN compute.** Time for the forward pass, loss computation, and backward pass.

3.7.2 *Query Execution Profiling*

Cypher queries are instrumented with the `PROFILE` keyword to collect operator-level statistics: rows produced, DB hits, and estimated memory usage. These are aggregated across batches and stored as both averaged CSV summaries and raw plan trees (`query_profile_raw.json`) for offline inspection.

3.7.3 *Python-Level Profiling*

Call-stack profiles are collected using Python's `cProfile` module and stored as `train_profile.txt`. These are used to identify hot-spots in the Python data pipeline that are not visible in the query-level metrics, such as tensor allocation overhead or unnecessary data copies.

3.7.4 *Sampler Comparison Framework*

A dedicated experiment runner executes multiple samplers under identical conditions and aggregates their results for side-by-side comparison. Plots comparing accuracy convergence, per-batch latency, and subgraph statistics are generated automatically at the end of the experiment.

3.8 CORRECTNESS AND VERIFICATION

The graph-database implementation is verified against the in-memory PyG baseline by comparing final test accuracy and convergence curves across multiple runs. Subgraph statistics (number of sampled nodes and edges per batch) are also checked to confirm that the samplers produce subgraphs of the expected size distribution.

A

APPENDIX

DECLARATION

Put your declaration here.

Put location here, February 2026

Harry Wik & Victor Pekkari

COLOPHON

This document was typeset using the typographical look-and-feel `classicthesis` developed by André Miede. The style was inspired by Robert Bringhurst's seminal book on typography "*The Elements of Typographic Style*". `classicthesis` is available for both LaTeX and LyX:

https://bitbucket.org/amiede/classicthesis/

Happy users of `classicthesis` usually send a real postcard to the author, a collection of postcards received so far is featured here:

http://postcards.miede.de/